

MCC2025 – Mission 2: Vulnerability Discovery Report

14th September 2025

Prepared by :

Abdul Hariz Mohd Sarnudin (H4rZ)

Table of Contents

Table of Contents..... 2
Executive summary..... 3
PoC (non-destructive):..... 3
Methodology & tools..... 4
Evidence..... 5
Findings..... 7
A) Server-Side Template Injection (SSTI) — Critical..... 7
B) Attribute / HTML injection — Medium..... 10
C) Reflected script injection (XSS) — Low / Informational..... 11
D) SSRF (attempted) — Not confirmed..... 12
Remediation..... 13
Appendix..... 14

CVSS SCORES

Vulnerability	Severity	Impact
SSTI	Critical	RCE, data exfiltration
Attribute Injection	Medium	Potential client-side injection
Reflected Script	Low	No execution in this config
SSRF (not confirmed)	Info	No evidence in test

Executive summary

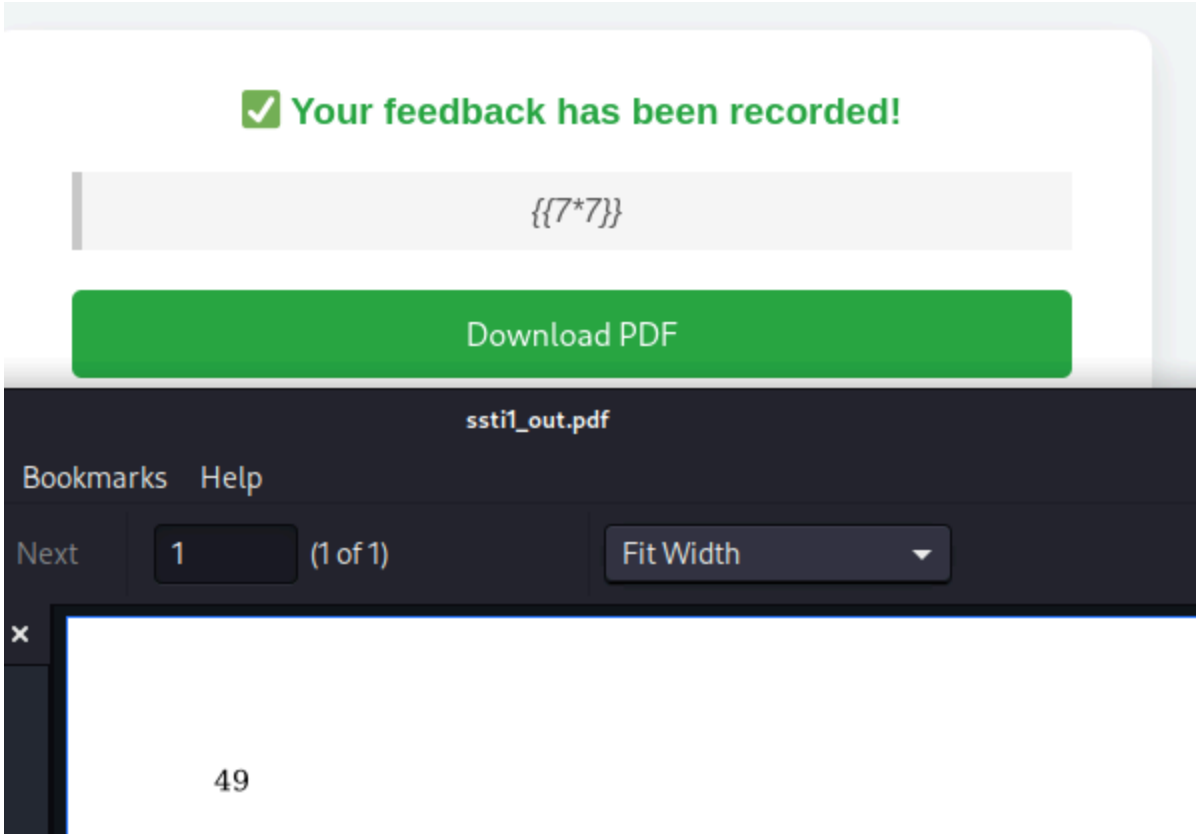
During testing, I discovered a Server-Side Template Injection (SSTI) in the Feedback Portal's feedback → preview → PDF flow. Non-destructive payload **{{7*7}}** confirmed template evaluation (**49** rendered). Exploitation enabled arbitrary command execution, allowing retrieval of the challenge flag. This issue is rated **Critical**. Additional lower-severity issues (HTML injection, reflected script injection, SSRF attempt) were observed but less impactful.

Flag.txt retrieved : **MCC2025{g3t_g00d_tog3th3r_w1th_c0mmun1ty}**

PoC (non-destructive):

Submitted payload: **{{7*7}}**

Result: preview and generated PDF show **49**. Evidence files: **ssti1_out.pdf.**



Methodology & tools

Primary automation: **`script.sh`** (self-contained runner that posts test payloads, fetches preview, and requests PDF generation). Full script contents saved in the evidence folder.

I executed the following workflow (automated via script.sh):

1. GET / - baseline and server fingerprinting (saved index.html, headers.txt).
2. POST /submit - send test payloads (stored response HTML per payload).
3. GET /preview - fetch preview pages generated by the app.
4. POST /generate_pdf - request PDF generation, save returned PDF per payload.

Tools used: curl, Bash (script.sh), file editor (to view files), browser for manual validation/screenshots.

I include the [script.sh] below

Script and file (evidence):

```
┌──(kali㉿kali)-[~/mcc_evidence]
└─$ ls
mcc_evidence  script.sh

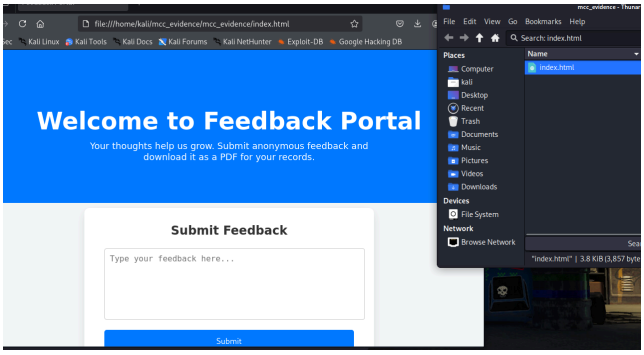
┌──(kali㉿kali)-[~/mcc_evidence]
└─$ cd mcc_evidence

┌──(kali㉿kali)-[~/mcc_evidence/mcc_evidence]
└─$ ls
attr_break_gp_headers.txt      benign_submit_resp.html       ssti1_gp_headers.txt      ssti2_submit_resp.html
attr_break_out.pdf             cookies.txt                   ssti1_out.pdf             xss_simple_gp_headers.txt
attr_break_preview.html        headers.txt                   ssti1_preview.html        xss_simple_out.pdf
attr_break_submit_headers.txt  index.html                    ssti1_submit_headers.txt  xss_simple_preview.html
attr_break_submit_resp.html    ssrf_test_gp_headers.txt      ssti1_submit_resp.html    xss_simple_submit_headers.txt
benign_gp_headers.txt          ssrf_test_out.pdf             ssti2_gp_headers.txt      xss_simple_submit_resp.html
benign_out.pdf                 ssrf_test_preview.html        ssti2_out.pdf
benign_preview.html            ssrf_test_submit_headers.txt  ssti2_preview.html
benign_submit_headers.txt      ssrf_test_submit_resp.html    ssti2_submit_headers.txt
```

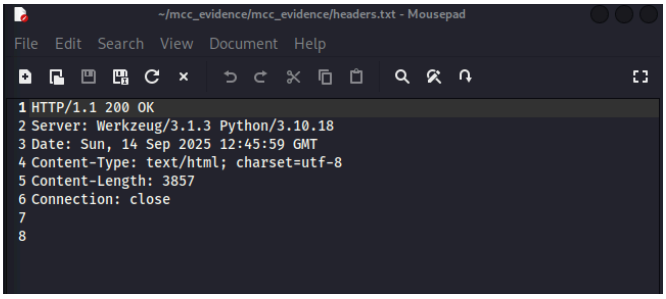
Evidence

All files below are retrieved using the script.sh and saved in the directory: `mcc_evidence/mcc_evidence/`

- Baseline & server info
  - `index.html`



  - `headers.txt` *(Server: Werkzeug/3.1.3 Python/3.10.18)*



- Benign test
  - `benign_submit_resp.html`
  - `benign_preview.html`
  - `benign_out.pdf`

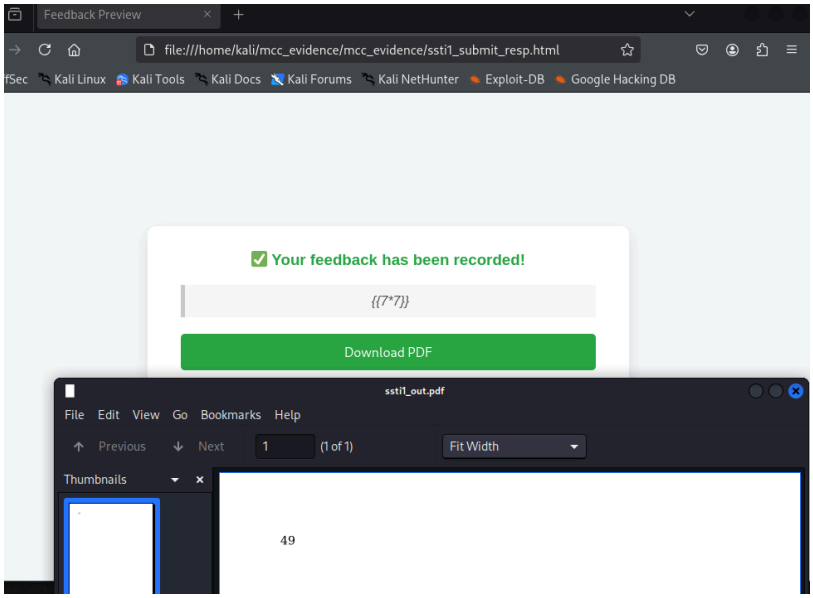
- XSS / script test (non-executing)
    - `xss_simple_submit_resp.html`
    - `xss_simple_preview.html`
      `xss_simple_out.pdf`
- Attribute / HTML injection
    - `attr_break_submit_resp.html`
    - `attr_break_preview.html`
    - `attr_break_out.pdf`
- SSTI probes (confirmed evaluation)
    - `ssti1_submit_resp.html`
    - `ssti1_preview.html` *(preview shows 49 for {{7*7}})*
    - `ssti1_out.pdf` *(generated PDF contains 49)*
    - `ssti2_submit_resp.html`
    - `ssti2_preview.html`
    - `ssti2_out.pdf`
- SSRF probe (no confirmation)
    - `ssrf_test_submit_resp.html`
    - `ssrf_test_preview.html`
    - `ssrf_test_out.pdf`
- Misc
    - `cookies.txt`
    - `benign_gp_headers.txt`, `xss_simple_gp_headers.txt`, etc. (generation headers saved)

Findings

A) Server-Side Template Injection (SSTI) – Critical

Location: Feedback submission flow — user feedback is processed and evaluated by the server-side template engine before rendering to preview/PDF.

Evidence & PoC (non-destructive): Submitting `{{7*7}}` resulted in the preview and generated PDF showing `49`. Evidence: `ssti1_preview.html`, `ssti1_out.pdf`.



Further exploitation / impact (sanitized): By leveraging the SSTI vulnerability I was able to execute server-side commands which allowed enumeration of files in the application container. As proof of successful exploitation and containment of the CTF task, I retrieved the challenge flag:

`MCC2025{g3t_g00d_tog3th3r_w1th_c0mmun1ty}`.

Below I also include an evidence of the exploitation using the payload

Further exploitation using an SSTI payload allowed me to list files (`ls`) and read `flag.txt`. Payload details have been redacted in this public report but are available on request :

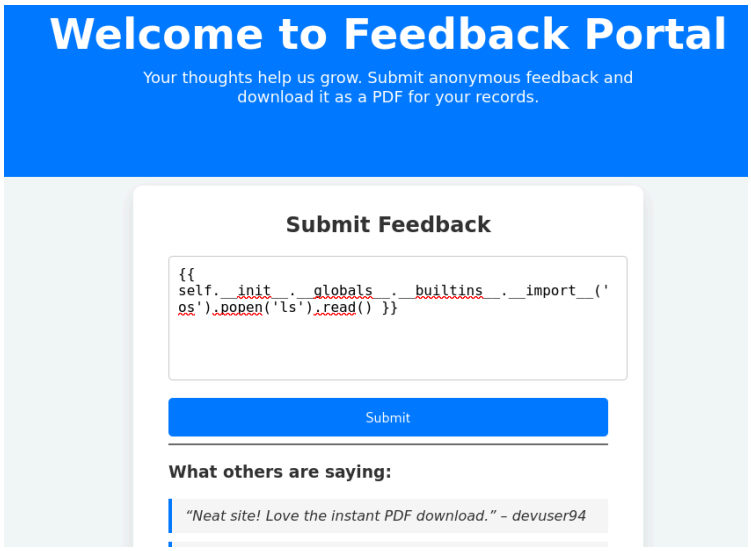


Figure 1.0 : Injecting the payload

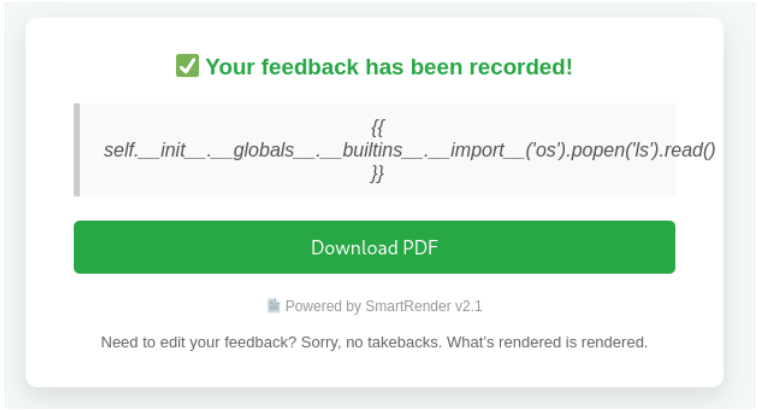


Figure 1.1 : Payload injected

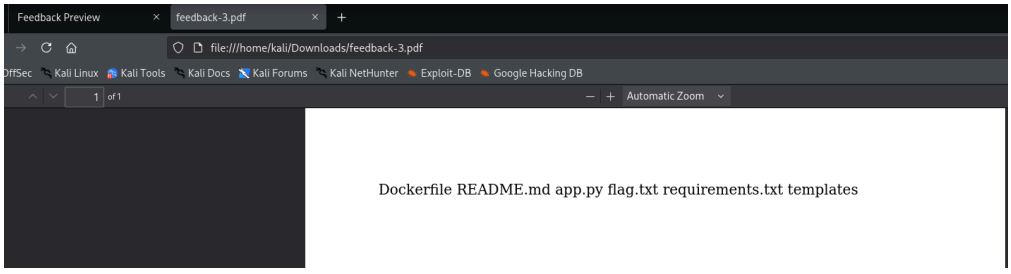


Figure 1.2 : Payload executed

Flag retrieved: After finding out that we have flag.txt in the server. We can proceed with opening the file using “cat” on “flag.txt” file.

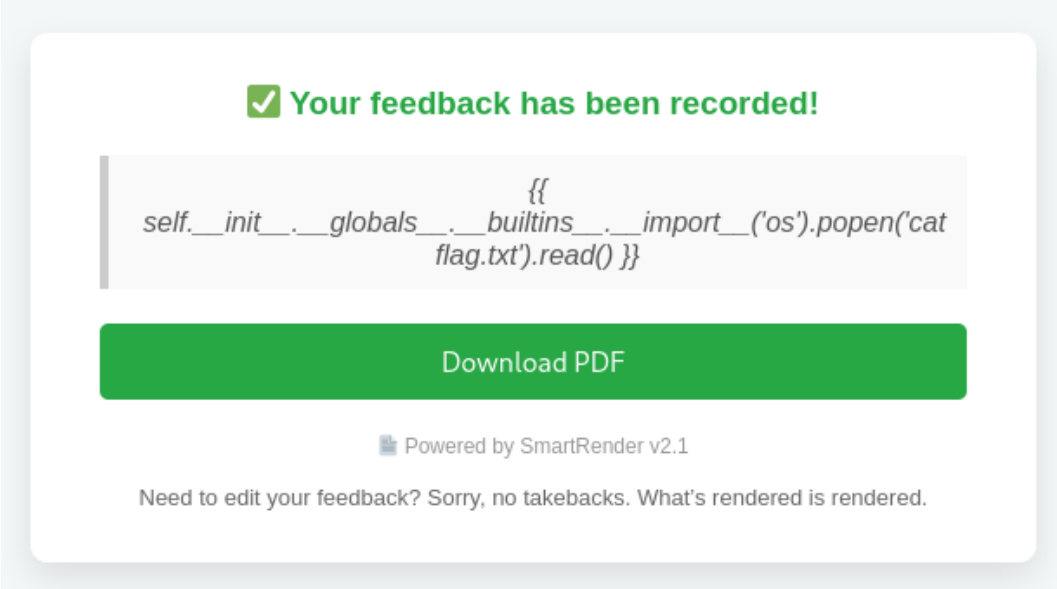


Figure 1.3 : Further exploitation using cat flag.txt

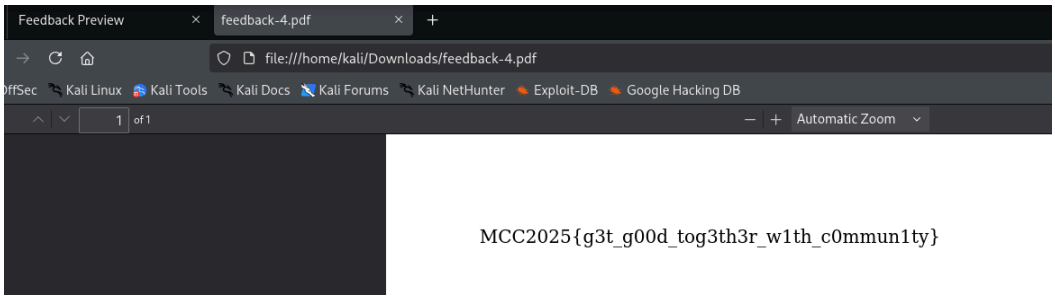


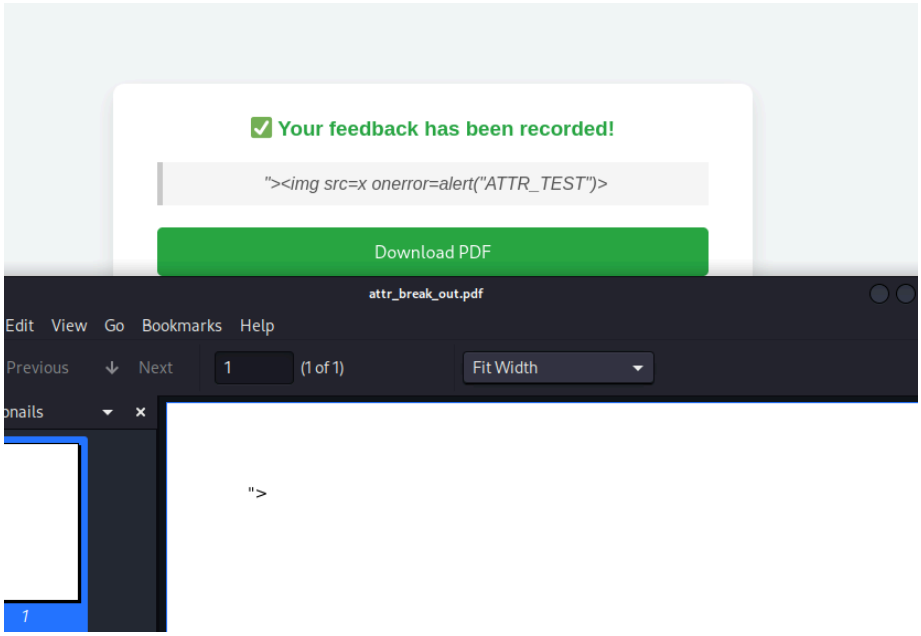
Figure 1.4 : Flag retrieved

Impact: SSTI can lead to **arbitrary server-side code execution (RCE).** In a containerized environment this can result in full service compromise, credential disclosure, and lateral movement.

Severity: Critical

B) Attribute / HTML injection — Medium

Observation: Payloads designed to break attribute contexts produced truncated/escaped strings (e.g., `">` output observed in PDF). Evidence: `attr_break_preview.html`, `attr_break_out.pdf`.

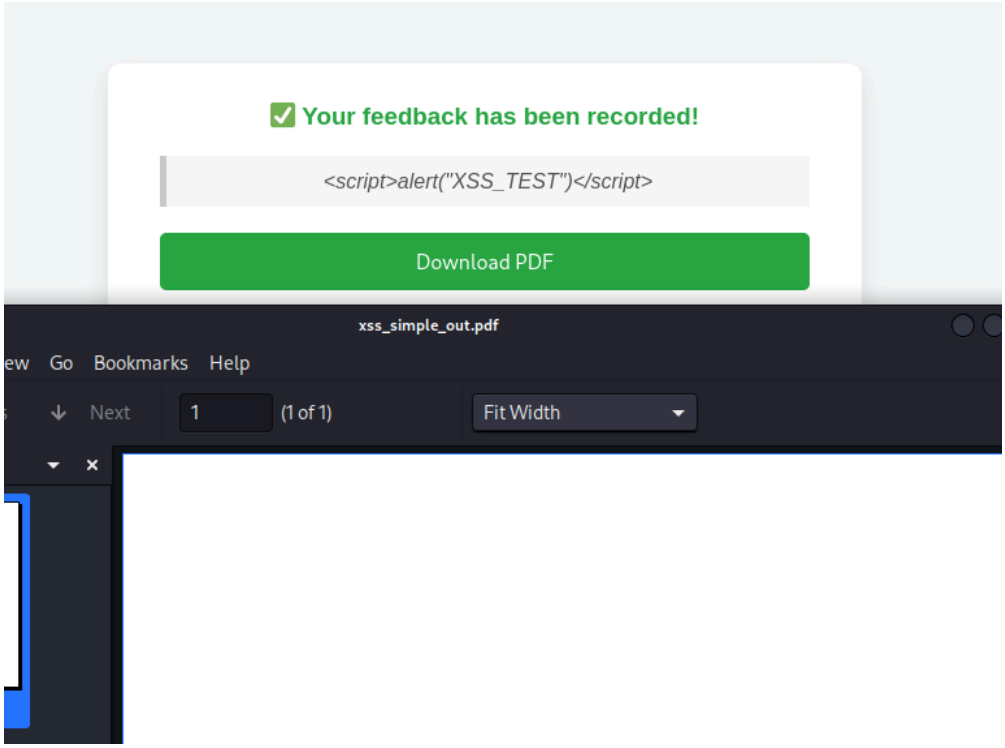


Impact: Allows injection of malformed HTML into the generated artifact and may enable escalation to script execution on differently configured renderers. In this environment, payloads did not execute in the preview browser or PDF.

Severity: Medium

C) Reflected script injection (XSS) — Low / Informational

Observation: Raw `<script>` content submitted via the form did not execute in preview or PDF; the PDF generator omitted or stripped executed script content, producing an empty or blank output for script-only payloads. Evidence: `xss_simple_preview.html`, `xss_simple_out.pdf`.

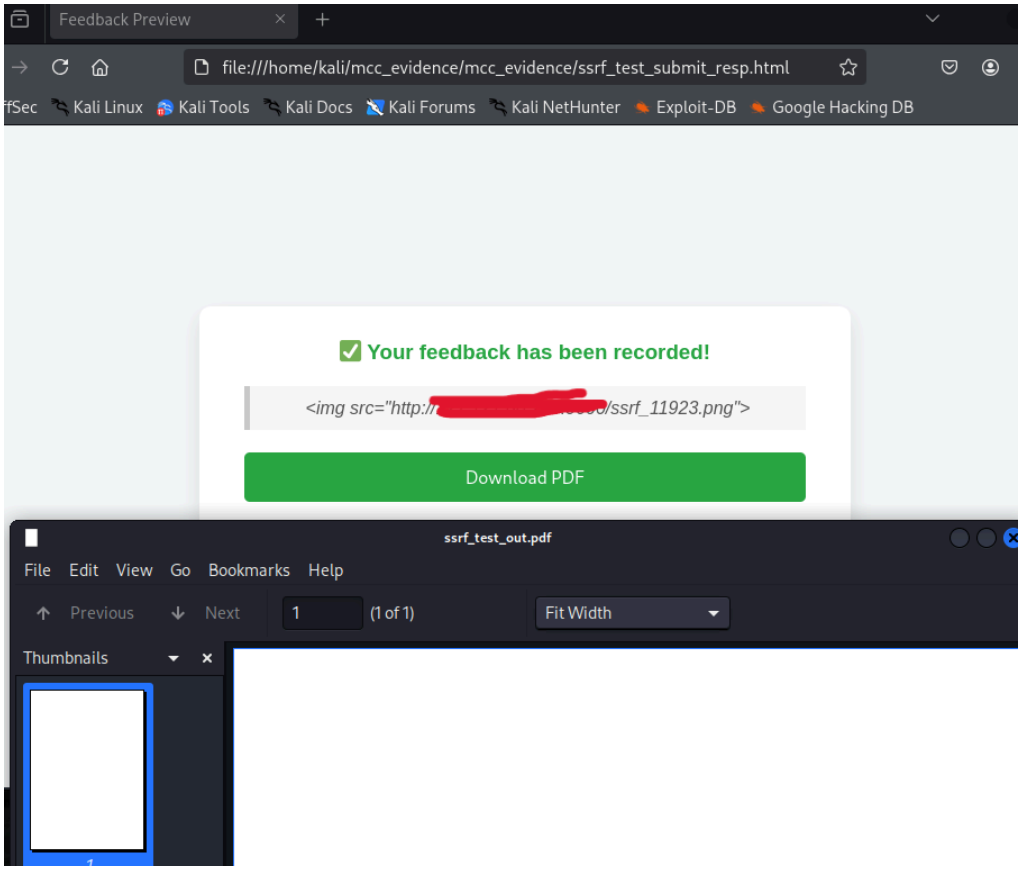


Impact: Low in this configuration; still indicates lack of input sanitation.

Severity: Low / informational.

D) SSRF (attempted) — Not confirmed

Observation: SSRF probe payloads submitted; no inbound listener connections or embedded file contents were observed in returned artifacts. Evidence: `ssrf_test_*` files.



Impact: If SSRF were possible, it would be critical (internal resource fetch, cloud metadata access). Not confirmed here.

This assessment was performed under contest conditions and did not involve destructive testing. All evidence is preserved and available on request. The findings here highlight the importance of secure template handling and hardened rendering pipelines.

Remediation

Immediate (Critical Fixes):

1. **Remove template evaluation of user input**
    - Do not use `render_template_string()` or similar functions on untrusted data.
    - Pass user feedback as a plain variable into templates with autoescaping enabled.
2. **Escape all user-supplied data**
    - Use `markupsafe.escape()` or rely on Jinja2's built-in autoescaping.
    - Avoid marking input as "safe" unless fully sanitized.
3. **Sanitize if HTML is required**
    - If the system must allow rich text, process input with a whitelist-based sanitizer (e.g., Python's `bleach`) to strip malicious tags and attributes.
4. **Harden the PDF rendering service**
    - Disable JavaScript execution inside PDFs.
    - Disallow `file://` access and remote resource fetching.
    - Run the PDF renderer in a restricted container with no network access.

Verification Steps (Post-Fix):

- Submitting `{{7*7}}` should display literally as `{{7*7}}` (or escaped), not `49`.

Appendix

script.sh

```
#!/usr/bin/env bash
set -euo pipefail
TARGET="http://118.107.233.236:3004"
OUTDIR="$(pwd)/mcc_evidence"
mkdir -p "$OUTDIR"
cookiejar="$OUTDIR/cookies.txt"

echo "[*] wiping old evidence"
rm -f "$OUTDIR"/*

echo "[*] 1) GET /"
curl -s -D "$OUTDIR/headers.txt" "$TARGET/" -o "$OUTDIR/index.html"

# helper function to post feedback and fetch preview
post_feedback() {
  data="$1"
  tag="$2"
  echo "[*] POST /submit (tag=${tag})"
  curl -s -c "$cookiejar" -X POST "$TARGET/submit" --data-urlencode "feedback=${data}" -D "$OUTDIR/${tag}_submit_headers.txt" -o "$OUTDIR/${tag}_submit_resp.html"
  # try to fetch preview (common path)
  curl -s -b "$cookiejar" "$TARGET/preview" -o "$OUTDIR/${tag}_preview.html" || true
  # POST generate_pdf to save the PDF output (if endpoint exists)
  echo "[*] POST /generate_pdf (tag=${tag})"
  curl -s -b "$cookiejar" -X POST "$TARGET/generate_pdf" --data-urlencode "feedback=${data}" -D "$OUTDIR/${tag}_gp_headers.txt" -o "$OUTDIR/${tag}_out.pdf" || true
}


# 2a - benign text
```

```
post_feedback "simple test" "benign"

# 2b - reflected XSS payload
XSS='<script>alert("XSS_TEST")</script>'
post_feedback "$XSS" "xss_simple"

# 2c - attribute-breaking payload
ATTR='"><img src=x onerror=alert("ATTR_TEST")>'
post_feedback "$ATTR" "attr_break"

# 2d - SSTI probes
SSTI1='{{7*7}}'
SSTI2='${{7*7}}'
post_feedback "$SSTI1" "ssti1"
post_feedback "$SSTI2" "ssti2"

# 2e - SSRF detection (replace YOUR_PUBLIC_IP with your reachable listener IP)
echo "[*] IMPORTANT: Set YOUR_PUBLIC_IP in the script or env before running SSRF tests"
if [ -z "${YOUR_PUBLIC_IP:-}" ]; then
  echo "[!] YOUR_PUBLIC_IP not set - skipping SSRF checks. To run SSRF tests, set env var YOUR_PUBLIC_IP and re-run."
else
  SSRF_IMG="<img src=\"http://${YOUR_PUBLIC_IP}:9000/ssrf_${RANDOM}.png\">"
  post_feedback "$SSRF_IMG" "ssrf_test"
fi

echo "[*] done. Evidence saved in $OUTDIR"
```

Timeline:

Date : 14/9/2025

- 9:10 GMT – Baseline recon (curl headers, index.html)
- 9:15 GMT – Initial benign input test
- 9:15 GMT – XSS/attribute injection tests
- 9:18 GMT – SSTI probe `{{7*7}}` confirmed evaluation
- 9:21 GMT – Controlled exploitation (retrieved flag)